

DFS :- (use stack)

```
void DFS (int v)
{
```

```
    int stack[10];
```

```
    int top = -1;
```

```
    stack[++top] = v;
```

```
    while (top >= 0)
```

```
    {
```

```
        int x = stack[top--];
```

```
        if (visited[x] == false)
```

```
        {
```

```
            visited[x] = true;
```

```
            cout << x;
```

```
            for (int i = 0; i < n-1; i++)
```

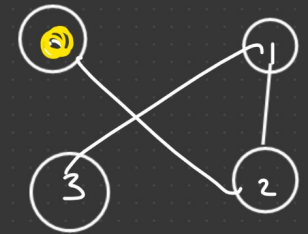
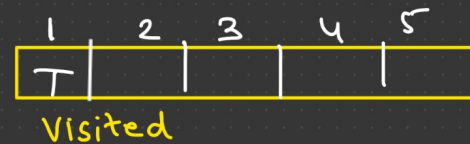
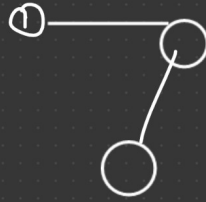
```
            {
```

```
                if (graph[x][i] && !visited[i])
```

```
                    stack[++top] = i;
```

```
            }
```

```
    }
```



→

	0	1	2	3
0	0	0	1	0
1	0	0	1	1
2	1	1	0	0
3	0	1	0	0

graph[0][i]

BFS :- (use queue)

```
void BFS ( int v )
```

```
{
```

```
    queue<int> q;
```

```
    q.push (v);
```

```
    visited [v] = true;
```

```
    while ( q.empty() == false )
```

```
    {
```

```
        int x = q.front();
```

```
        q.pop();
```

```
        cout << x; ✓
```

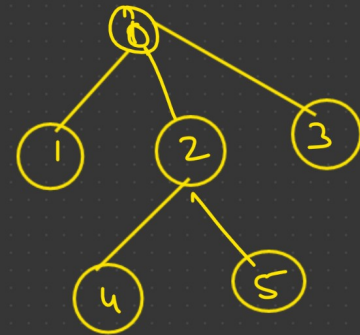
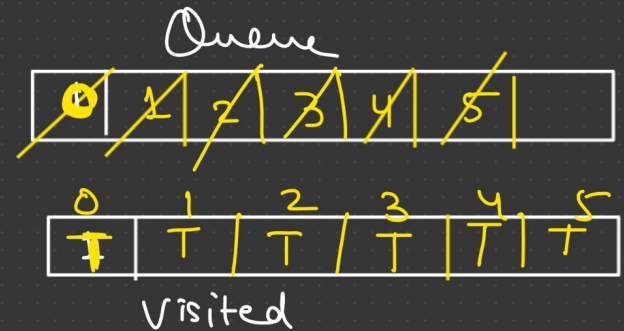
```
        for ( int i = 0; i < n; i++ )
```

```
        { if ( graph[x][i] && !visited[i] )
```

```
            { q.push(i); visited[i] = true;
```

```
            }
```

```
        }
```



0, 1, 2, 3, 4, 5

Dijkstra's Algorithm

* Single Source Shortest path Algorithm.

0	1	2	3	4
T	T	T	T	T

visited

0	1	2	3	4
0	2	3	6	5

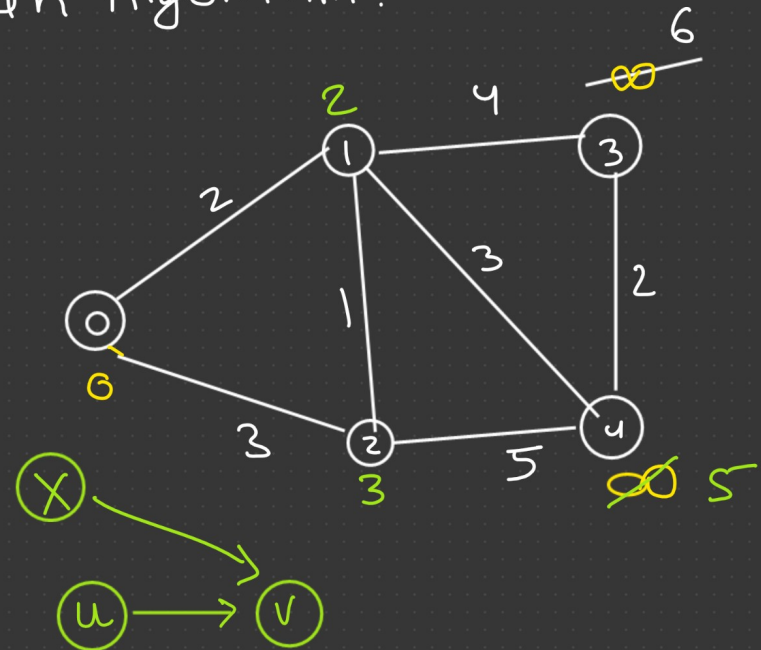
distance

0	1	2	3	4
-1	0	0	1	1

predecessor

0 $\rightarrow$ 4

0 $\rightarrow$ 1 $\rightarrow$ 4
2 + 3 = 5



if (graph[u][v] & !visited[v]) $\rightarrow$ ①

if (dist[v] > graph[u][v] + dist[u])
 {
 dist[v] = graph[u][v] + dist[u];
 }

$$\text{graph}[u][v] = 5$$

$$\text{source} \rightarrow \underline{\text{dist}}[v] = 50$$

$$\text{source} \rightarrow \underline{\text{dist}}[u] = 20$$

$$\text{Source} \quad \text{dist}[u] + \text{graph}[u][v] = \text{dist}[v]$$



$$\text{dist}[\text{dost}] = 10$$

$$\text{dist}[\text{clg}] = 50$$

$$\text{graph}[\text{dost}][\text{clg}] = 40$$

$$\text{dist}[\text{clg}] > \text{dist}[\text{xyz}] + \text{graph}[\text{xyz}][\text{clg}]$$

$$\text{dist}[\text{clg}] = \text{dist}[\text{xyz}] + \text{graph}[\text{xyz}][\text{clg}]$$

```
void dijkstra( int v)
```

```
{  
    int dist[10];  
    bool visited[10];  
    int predecessor[10];
```

```
    for (int i = 0 ; i < n ; i++)  
    {  
        dist[i] = INF;  
        visited[i] = false;  
        predecessor[i] = -1;  
    }  
}
```

```
    dist[v] = 0  
    visited[v] = True;  
    predecessor[v] = -1;
```

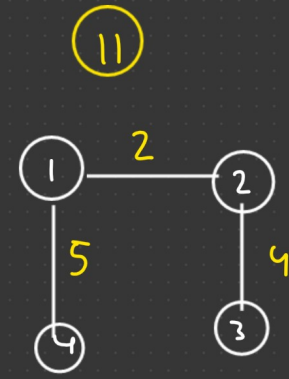
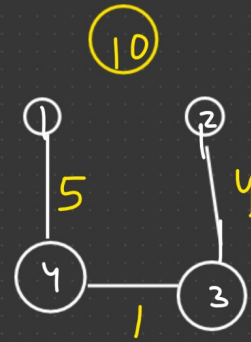
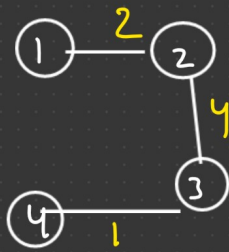
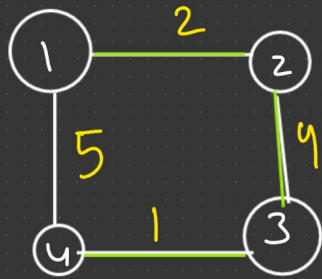
```
    for (int i = 0 ; i < n ; i++)
```

```
    {  
        if (graph[v][i] && !visited[i] &&  
            dist[i] > dist[v] + graph[v][i])  
        {  
            dist[i] = dist[v] + graph[v][i];  
            predecessor[i] = v;  
            visited[i] = true;
```

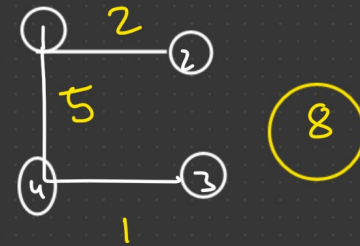
```
    }  
}
```

MST (Minimum Spanning Tree) (greedy Approach)

MST (Minimum Spanning Tree) (greedy Approach)

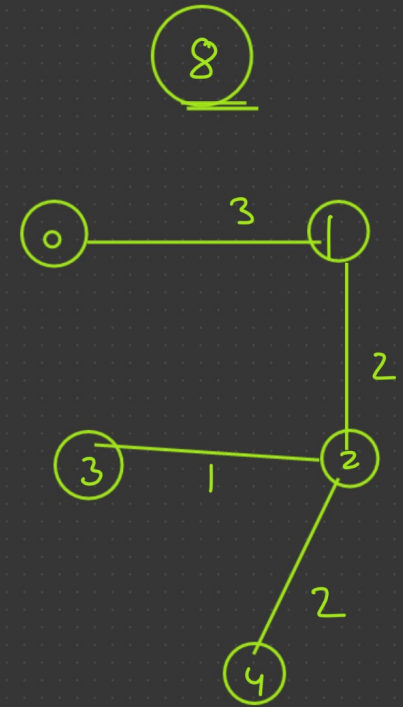
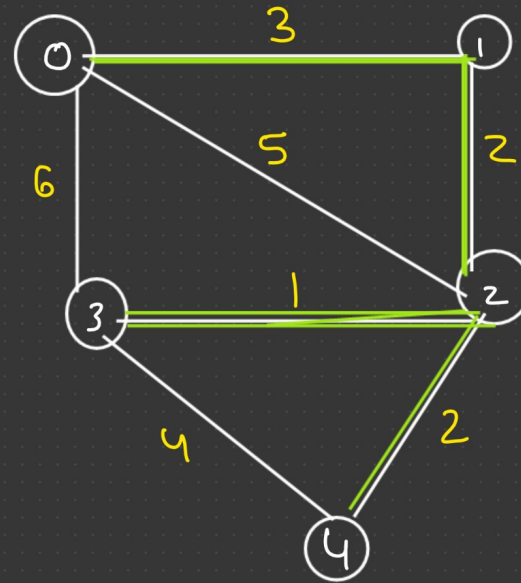


$$\text{No. of vertex} - 1 = \text{No. of edge}$$



Prim's Algorithm

- ① Prim's Code
- ② Krushkal's Algo.



Sorting

① Bubble Sort:-

swap
4, 2, 6, 1, 5, 8

2, 4, 6, 1, 5, 8
swap

2, 4, 1, 6, 5, 8
swap

2, 4, 1, 5, 6, 8

increasing

1st iteration

2, 4, 1, 5, 6, 8
swap

2, 1, 4, 5, 6, 8

2nd iteration

2, 1, 4, 5, 6, 8

1, 2, 4, 5, 6, 8

✓

10 person $\rightarrow$ It is sufficient to arrange 9 person.

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

n -element $\rightarrow n-1$ iteration is sufficient.

void bubblesort()

```
{  
  int a[] = { 2, 6, 4, 3, 1, 8 };  
             i   j       i
```

```
  int n = 6;
```

```
  for (int i = 0; i < n-1; i++)
```

```
  {  
    for (int j = 0; j < n-i-1; j++)  
    {
```

```
      if (a[j] > a[j+1])
```

```
      {
```

```
        int temp = a[j];
```

```
        a[j] = a[j+1];
```

```
        a[j+1] = temp;
```

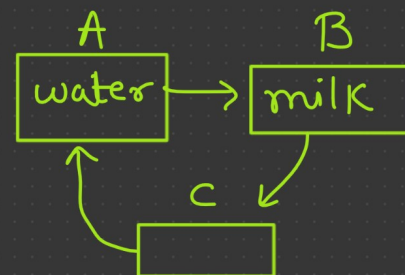
```
      }  
    }  
  }
```

2	6	4	3	1	8
0	1	2	3	4	5

$j=5$ $i=0$

$j=4$ $i=1$

$j=3$ $i=2$



$C = B;$

$B = A;$

$A = C;$

$temp = X$

$x = y;$

$y = temp$

}

$O(n^2)$

```
for (int i=0; i<n-1; i++)  
{  
    for (int j=0; j<n-i-1; j++)  
    {  
        if (a[j] > a[j+1])  
        {  
            int temp = a[j];  
            a[j] = a[j+1];  
            a[j+1] = temp;  
        }  
    }  
}
```

$i=0$

$j=0 \longrightarrow n-1$

$i=1$

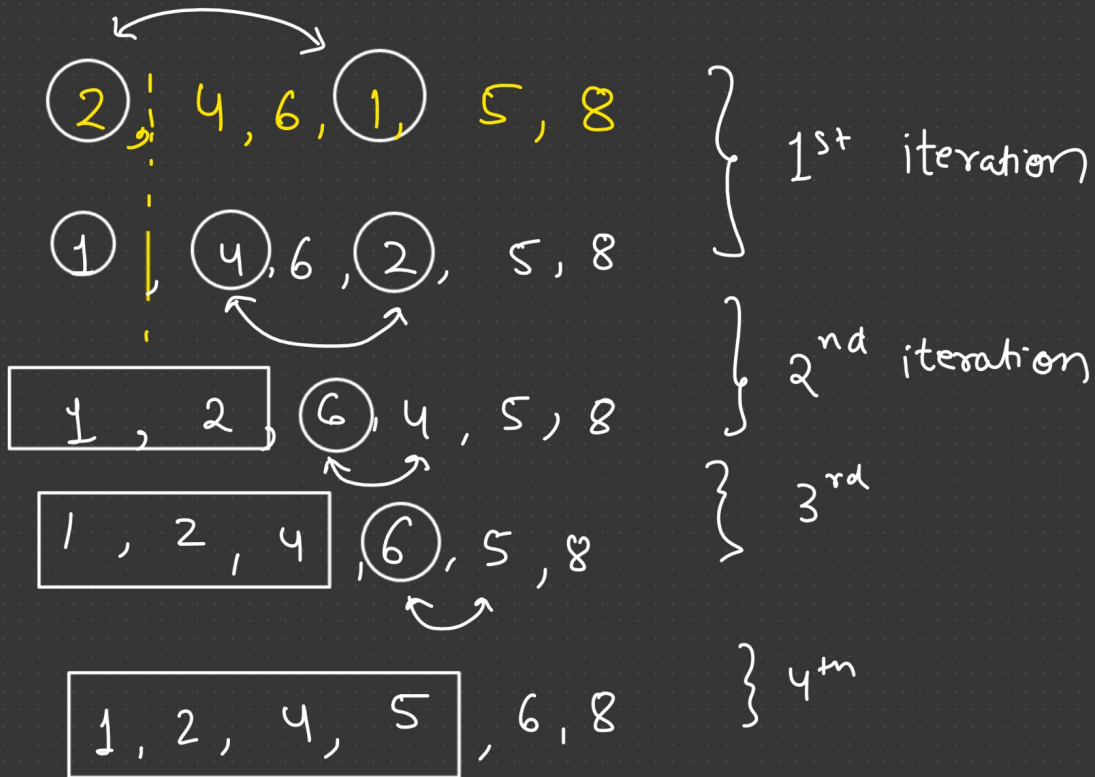
$j=0 \longrightarrow n-2$

$i=2$

$j=0 \longrightarrow n-3$

Selection Sort

min = ~~2~~ 1



```

void selectionSort()
{
    int a[] = { 2, 4, 6, 1, 5, 8 };
    int n = 6;

    for (int i = 0 ; i < n - 1 ; i++)
    {
        int min = i;

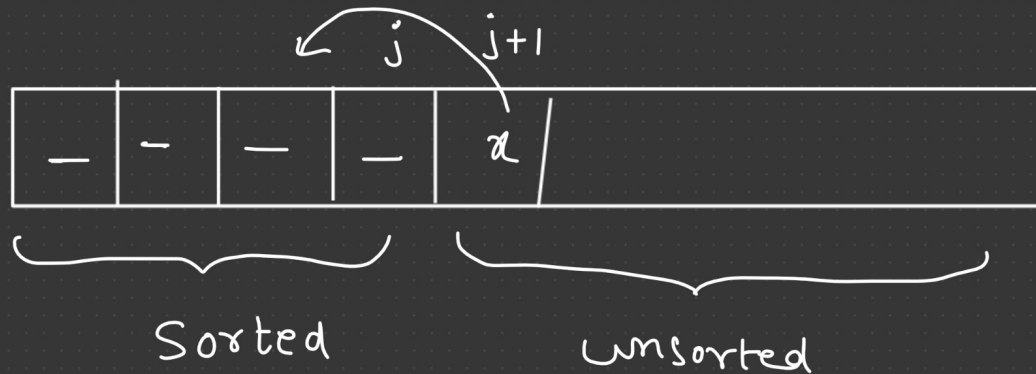
        for (int j = i + 1 ; j < n ; j++)
        {
            if ( a[j] < a[min] )
                min = j;
        }

        swap( a[i], a[min] );
    }
}

```

temp =
 -
 -

Insertion Sort



2, 4, 6, 1, 5, 8, 3

2, 4, 6, 1, 5, 8, 3

2, 4, 6, 1, 5, 8, 3

1, 2, 4, 6, 5, 8, 3

1, 2, 4, 5, 6, 8, 3

1, 2, 4, 5, 6, 8, 3

1, 2, 3, 4, 5, 6, 8

```
void insertionSort()
```

```
{ for (int i = 1; i < n; i++)
```

```
{ int key = a[i];
```

```
int j = i - 1; j = 3
```

```
while (j >= 0 && a[j] > key)
```

```
{ a[j+1] = a[j];
```

```
  j--;
```

```
  a[j+1] = key;
```

$O(n^2)$

```
}
```